Advanced Rust 2026 - Lab 4: Async Rate Limiter Library - Reference Solution

Lukáš Hozda

Spring 2026

Cargo.toml

```toml
[package]
name = "async_rate_limiter"
version = "0.1.0"
edition = "2021"

[dependencies]
tokio = { version = "1", features = ["macros", "rt-multi-thread", "time", "test-util"] }
```

src/lib.rs

```rust
use std::collections::hash_map::Entry;
use std::collections::HashMap;
use std::fmt;
use std::sync::{Arc, Mutex};
use std::time::Duration;
use tokio::time::{sleep, timeout, Instant};

#[derive(Debug, Clone, PartialEq, Eq)]
pub enum Error {
    Timeout,
    InvalidConfig,
}

impl fmt::Display for Error {
    fn fmt(&self, f: &mut fmt::Formatter<'_>) -> fmt::Result {
        match self {
            Self::Timeout => f.write_str("rate limit exceeded and timed out waiting"),
            Self::InvalidConfig => f.write_str("invalid rate limiter configuration"),
        }
    }
}

impl std::error::Error for Error {}

/// A simple token-bucket rate limiter for async Tokio code.
pub struct RateLimiter {
    permits_per_second: f64,
    burst_capacity: f64,
    state: Mutex<RateLimiterState>,
}

#[derive(Debug)]
struct RateLimiterState {
    tokens: f64,
    last_refill: Instant,
}

impl RateLimiter {
    /// Creates a new rate limiter with a full initial bucket.
    pub fn new(permits_per_second: f64, burst_capacity: u32) -> Result<Self, Error> {
```

```
        if !permits_per_second.is_finite() || permits_per_second <= 0.0 || burst_capacity == 0 {
            return Err(Error::InvalidConfig);
        }

        Ok(Self {
            permits_per_second,
            burst_capacity: burst_capacity as f64,
            state: Mutex::new(RateLimiterState {
                tokens: burst_capacity as f64,
                last_refill: Instant::now(),
            }),
        })
    }

    /// Tries to acquire one permit immediately.
    pub fn try_acquire(&self) -> bool {
        let mut state = self.state.lock().unwrap();
        self.refill(&mut state, Instant::now());
        if state.tokens >= 1.0 {
            state.tokens -= 1.0;
            true
        } else {
            false
        }
    }

    /// Waits until a permit becomes available.
    pub async fn acquire(&self) -> Result<(), Error> {
        loop {
            if self.try_acquire() {
                return Ok(());
            }

            sleep(self.time_until_next_token()).await;
        }
    }

    /// Waits until a permit becomes available or the timeout expires.
    pub async fn acquire_timeout(&self, timeout_duration: Duration) -> Result<(), Error> {
        timeout(timeout_duration, self.acquire())
            .await
            .map_err(|_| Error::Timeout)?
    }

    fn time_until_next_token(&self) -> Duration {
        let mut state = self.state.lock().unwrap();
        self.refill(&mut state, Instant::now());

        if state.tokens >= 1.0 {
            return Duration::from_secs(0);
        }

        let missing = 1.0 - state.tokens;
        Duration::from_secs_f64(missing / self.permits_per_second)
    }

    fn refill(&self, state: &mut RateLimiterState, now: Instant) {
        let elapsed = now.duration_since(state.last_refill).as_secs_f64();
        let replenished = state.tokens + elapsed * self.permits_per_second;
        state.tokens = replenished.min(self.burst_capacity);
        state.last_refill = now;
    }
}

/// Registry of named rate limiters.
```

```rust
pub struct RateLimiterRegistry {
    inner: Mutex<HashMap<String, Arc<RateLimiter>>>,
}

impl RateLimiterRegistry {
    /// Creates a new empty registry.
    pub fn new() -> Self {
        Self {
            inner: Mutex::new(HashMap::new()),
        }
    }

    /// Registers or replaces a limiter under `name` and returns the stored limiter.
    pub fn register<S: Into<String>>(&self, name: S, limiter: RateLimiter) -> Arc<RateLimiter> {
        let name = name.into();
        let limiter = Arc::new(limiter);
        self.inner.lock().unwrap().insert(name, limiter.clone());
        limiter
    }

    /// Retrieves a limiter by name.
    pub fn get<S: AsRef<str>>(&self, name: S) -> Option<Arc<RateLimiter>> {
        self.inner.lock().unwrap().get(name.as_ref()).cloned()
    }

    /// Returns an existing limiter or creates a new one atomically.
    pub fn get_or_create<S, F>(&self, name: S, create_fn: F) -> Arc<RateLimiter>
    where
        S: Into<String>,
        F: FnOnce() -> RateLimiter,
    {
        let name = name.into();
        let mut inner = self.inner.lock().unwrap();

        match inner.entry(name) {
            Entry::Occupied(entry) => entry.get().clone(),
            Entry::Vacant(entry) => {
                let limiter = Arc::new(create_fn());
                entry.insert(limiter.clone());
                limiter
            }
        }
    }

    /// Removes a limiter by name.
    pub fn remove<S: AsRef<str>>(&self, name: S) -> Option<Arc<RateLimiter>> {
        self.inner.lock().unwrap().remove(name.as_ref())
    }
}

#[cfg(test)]
mod tests {
    use super::*;
    use tokio::task::yield_now;
    use tokio::time::advance;

    #[tokio::test(start_paused = true)]
    async fn try_acquire_consumes_burst() {
        let limiter = RateLimiter::new(2.0, 2).unwrap();
        assert!(limiter.try_acquire());
        assert!(limiter.try_acquire());
        assert!(!limiter.try_acquire());
    }

    #[tokio::test(start_paused = true)]
```

```rust
    async fn acquire_waits_for_refill() {
        let limiter = Arc::new(RateLimiter::new(1.0, 1).unwrap());
        assert!(limiter.try_acquire());

        let cloned = limiter.clone();
        let waiter = tokio::spawn(async move { cloned.acquire().await });

        yield_now().await;
        assert!(!waiter.is_finished());

        advance(Duration::from_millis(999)).await;
        yield_now().await;
        assert!(!waiter.is_finished());

        advance(Duration::from_millis(1)).await;
        yield_now().await;
        assert_eq!(waiter.await.unwrap(), Ok(()));
    }

    #[tokio::test(start_paused = true)]
    async fn acquire_timeout_fails_when_wait_too_long() {
        let limiter = RateLimiter::new(1.0, 1).unwrap();
        assert!(limiter.try_acquire());

        let result = limiter.acquire_timeout(Duration::from_millis(500)).await;
        assert_eq!(result, Err(Error::Timeout));
    }

    #[test]
    fn invalid_config_is_rejected() {
        assert!(matches!(RateLimiter::new(0.0, 1), Err(Error::InvalidConfig)));
        assert!(matches!(RateLimiter::new(f64::NAN, 1), Err(Error::InvalidConfig)));
        assert!(matches!(RateLimiter::new(1.0, 0), Err(Error::InvalidConfig)));
    }

    #[test]
    fn registry_basic_operations() {
        let registry = RateLimiterRegistry::new();
        assert!(registry.get("login").is_none());

        let first = registry.register("login", RateLimiter::new(5.0, 1).unwrap());
        let fetched = registry.get("login").unwrap();
        assert!(Arc::ptr_eq(&first, &fetched));

        let second = registry.register("login", RateLimiter::new(10.0, 2).unwrap());
        let fetched = registry.get("login").unwrap();
        assert!(Arc::ptr_eq(&second, &fetched));
        assert!(!Arc::ptr_eq(&first, &second));

        let removed = registry.remove("login").unwrap();
        assert!(Arc::ptr_eq(&second, &removed));
        assert!(registry.get("login").is_none());
    }

    #[test]
    fn get_or_create_is_stable_for_existing_key() {
        let registry = RateLimiterRegistry::new();
        let first = registry.get_or_create("api", || RateLimiter::new(3.0, 1).unwrap());
        let second = registry.get_or_create("api", || RateLimiter::new(99.0, 9).unwrap());
        assert!(Arc::ptr_eq(&first, &second));
    }
}
```